Senior Software Engineer Exercise

Business Scenario

Your team is developing software for an industrial manufacturing machine used in automated production environments.

The machine executes a production workflow that consists of multiple processing stages (stage_1, stage_2, stage_3). Stages may be executed in parallel based on the current value of the sensors if resources are available. Sensor values are often updated every 100ms. Note that the manufacturing process is expected to evolve over time. Therefore, the software should be designed with an acceptable level of flexibility and maintainability in mind.

List of sensors include:

- Temperature sensors
- Pressure sensors

List of resources:

- Resource R_A
- Resource R_B
- Resource R_C

Stage Map:

- stage_1 {R_A, R_B}
- stage_2 {R_C, R_B}
- stage_3 {R_A, R_C}

If Temperature is more than 10.0 and Pressure is less than 100 then stage_1 and stage_2 shall be executed for a t1 period.

If Temperature is more than 5.0 and Pressure is less than 50 then stage_3 and stage_2 shall be executed for a t2 period.

If Temperature is more than 20.0 and Pressure is less than 100 then stage_1 and stage_3 shall be executed for a t3 period.

- Design constraint 1: t1, t2, and t3 will be random durations varying from 100ms to 5s
- Design constraint 2: A resource will have busy, idle, and error states.
- Design constraint 3: New sensors may be introduced.
- Design constraint 4: Existing sensors may be replaced.

- Design constraint 5: Conflicting stages (competition on resource allocation) should be addressed with FIFO.
- Design constraint 6: Each resource can be occupied for a minimum of 50ms and a maximum of 1s.

Must Have

Preventing concurrency-related issues is a must-have part of this exercise. Therefore, consider tackling concurrency challenges when designing the solution. The proposed design should demonstrate a good understanding of concurrency problems and offer mechanisms to address at least two potential concurrency-related problems. A successful candidate will be able to present at least one deadlock problem and one non-deadlock problem.

Out of Scope

Following topics are out of scope:

- Handling error state
- Inter-process communication
- Sensor identity and units
- Idempotence

Part 1 – Conceptual Design

Before implementation, design the solution at a conceptual level.

Describe:

Architecture

- Main system components
- Component responsibilities
- Communication patterns

Our preferred formats are:

- Markdown (`.md`)
- Mermaid diagrams

However, candidates are free to use any tools they consider appropriate.

Part 2 – Implementation

Implement the solution based on your design.

Required Capabilities

At minimum, the solution should demonstrate:

- Sensor registration or management
- Reception of sensor measurements
- Support for multiple independent consumers (e.g., threads or processes) of the same sensor data
- Properly manage your resources (A, B, C) among different consumers and handle concurrency probelms

Technology Choices

Your main programming language must be either C or C#, our perference will be C#.

- Programming language
- Frameworks
- Storage technologies
- Architectural style

Document the decisions you make and any assumptions you consider important.

Submission Requirements

The completed exercise must be submitted as a GitHub repository.

The repository should contain:

- Source code
- Design documentation
- Architecture diagrams
- Tests
- Build and execution instructions
- Any additional supporting material

Please provide a link to the repository containing your solution at least 24h before your video interview.

Evaluation Criteria

The solution will be evaluated based on:

- Architecture and design quality
- Correctness
- Simplicity and clarity
- Extensibility
- Maintainability
- Testability
- Documentation quality
- Commit history and development process

- Ability to explain design decisions and trade-offs
- Orginality of the work

The goal is to demonstrate engineering judgment, not simply produce a working application or an over engineered solution.

Submission Requirements

The completed exercise must be delivered as a GitHub repository.

Please provide:

- A link to the GitHub repository (public).
- All source code.
- Architecture and design documentation.
- Diagrams and other design artifacts.
- Build and execution instructions.
- Automated tests.

All deliverables must be stored within the repository itself. External documents, files, or links should be avoided unless absolutely necessary. Once we have cloned the repository, the candidate MUST remove it from their git and must not disclose any information publicly.

Use of AI and External Resources

Candidates are free to use any available resources, including:

- AI-assisted development tools
- Online documentation
- Open-source libraries and frameworks

The use of such tools is not prohibited and will not negatively impact the evaluation. However, during the review process, candidates should be prepared to discuss and explain:

- Architectural decisions
- Design trade-offs
- Domain modeling choices
- Implementation details
- Testing strategy
- Technology selections

The expectation is that candidates fully understand the solution they submit. If a candidate is unable to adequately explain significant parts of the design or implementation, it may be concluded that:

- The work was not primarily produced by the candidate; or
- The solution relies excessively on generative AI without sufficient understanding.

In such cases, the application may be rejected.

Last but not least, we value engineering judgment and understanding more than the volume of code produced.